Taxi-v4 — Q-Learning tabular

LunarLander-v3 — SARSA vs Q-Learning con discretización

Car Racing con Gymnasium y PPO

Aplicación de *PPO* (*Proximal Policy Optimization*) al entorno *CarRacing-v3*.

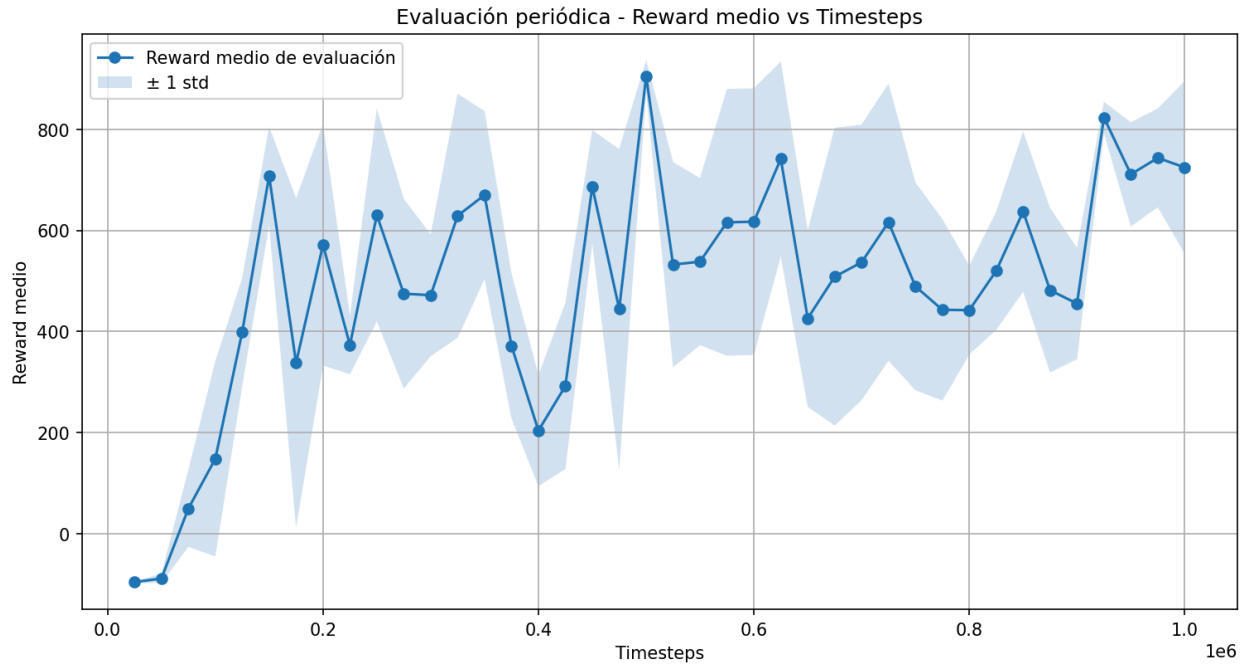
Entre los problemas que trabajamos incluimos **Car Racing** porque representa un problema de aprendizaje por refuerzo completo: el agente debe **aprender a conducir a partir de imágenes**, tomando acciones continuas para maximizar una recompensa acumulada.

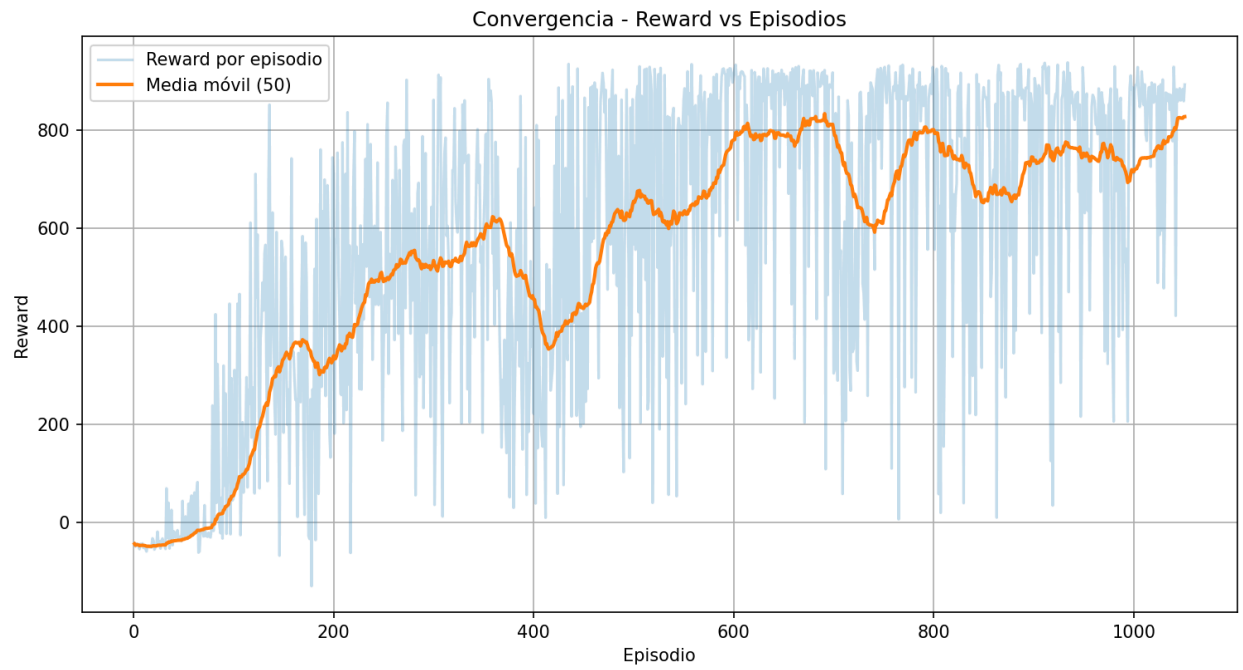
El problema consiste en entrenar un agente de aprendizaje por refuerzo capaz de conducir en **CarRacing-v3** a partir de observaciones visuales, tomando acciones continuas para controlar dirección, aceleración y freno. Se trata de un problema exigente porque combina **percepción visual**, **control continuo**, **estabilidad temporal** y **generalización** a distintas pistas. La solución se abordó utilizando **Gymnasium** como framework del entorno y **PPO** como algoritmo base, implementado con **Stable-Baselines3**. A partir de una configuración inicial con imágenes en grayscale, frame stacking y política convolucional, se construyó un baseline y luego se fue iterando experimentalmente sobre distintas variantes, modificando de a un parámetro por vez para poder analizar con claridad el efecto de cada cambio.

Durante el trabajo aparecieron varios inconvenientes. En primer lugar, hubo dificultades técnicas de implementación y compatibilidad, especialmente en **Windows con VS Code**, relacionadas con serialización, vectorización de entornos, formatos de observación y consistencia entre entrenamiento y evaluación; estos problemas se resolvieron corrigiendo wrappers, ajustando el pipeline visual y asegurando que train y eval usaran exactamente el mismo espacio de observación. En segundo lugar, aparecieron problemas propios del entrenamiento: algunas configuraciones lograban rewards altos pero generaban comportamientos incorrectos, como salirse de pista o volver en sentido contrario. Esto mostró que el reward por sí solo no alcanzaba para validar la calidad de la política, por lo que fue necesario complementar la evaluación cuantitativa con análisis cualitativo mediante videos y métricas adicionales como **completion rate**. A nivel experimental, se probaron distintos ajustes de PPO; varios no mejoraron o incluso empeoraron el resultado, mientras que la mejor mejora surgió al **reducir el tamaño de la observación mediante resize a (84,84)**, lo que permitió obtener una política más estable y una conducción visualmente superior. En síntesis, el problema se resolvió mediante un proceso iterativo de experimentación controlada, donde cada dificultad fue guiando ajustes concretos tanto en la implementación como en la formulación de la observación y la evaluación.

Cada folder de la solución tiene el detalle de lo ejecutado, los cambios, la explicación de cada corrida, etc.

Gráficos:





Código más relevante:

```
def main() -> None:
    cfg = get_config()
    set_global_seeds(cfg.seed)
    paths = make_run_dirs(cfg)
    export_config(cfg, paths)
```

```python
print("=" * 80)
print("CarRacing-v3 PPO baseline")
print(f"Run dir:          {paths['base']}")
print(f"Seed:             {cfg.seed}")
print(f"Total timesteps:  {cfg.total_timesteps}")
print(f"Parallel envs:    {cfg.n_envs}")
print(f"Eval every:       {cfg.eval_freq_steps} env steps")
print(f"Checkpoint every: {cfg.checkpoint_freq_steps} env steps")
print("=" * 80)

train_env = make_train_vec_env(cfg, paths)
eval_env = make_eval_vec_env(cfg, paths)

model = build_model(cfg, train_env, paths)
callbacks = build_callbacks(cfg, eval_env, paths)

try:
    model.learn(
        total_timesteps=cfg.total_timesteps,
        callback=callbacks,
        progress_bar=True,
        tb_log_name="ppo_carracing",
        reset_num_timesteps=True,
    )

    final_model_path = paths["base"] / "final_model.zip"
    model.save(str(final_model_path))
    print(f"Saved final model to: {final_model_path}")
    print(f"Best model path:      {paths['best_model']}")
    print(f"TensorBoard log dir:  {paths['tb']}")

finally:
    train_env.close()
    eval_env.close()
```